

Modelo de Ising 2D: Monte Carlo y Transición de Fase

Julián Rojas^{1*}

¹ *Departamento de Física, Universidad Técnica Federico Santa María*

6 de junio de 2026

Resumen

Se presenta una simulación del modelo de Ising bidimensional basada en métodos markovianos. Primero se simuló el sistema usando el método Metropolis-MonteCarlo y luego se implementó una mejora usando el formalismo de Wolf para el tratamiento en bloque. Finalmente, se calcularon las cantidades macroscópicas de interés $\langle M \rangle$, $\langle \chi \rangle$ y $\langle C \rangle$ para diferentes temperaturas para así estudiar su evolución. Además, se añadió un remuestreo con bootstrap para cuantificar el error de estas mediciones macroscópicas obtenidas usando MonteCarlo.

1. Introducción

Históricamente la humanidad se ha maravillado por las propiedades magnéticas de la materia. Naturalmente, esto llevó a los físicos a buscar una forma satisfactoria que explique este fenómeno. En vista del fallo de la electrodinámica clásica en describir correctamente este fenómeno se tuvo que tomar en consideración fenómenos cuánticos de la materia. Bajo el alero de la sombra del spin como un fenómeno intrínsecamente relacionado a un comportamiento magnético es que surge el modelo de Ising. Este buscó explicar como ciertos materiales son capaces de adquirir una magnetización a determinada temperatura, es decir estudiar el ferromagnetismo. Bajo una descripción mecanoestadística del fenómeno se pretende entonces describir el origen de las transiciones de fase magnéticas de la materia. Cabe aclarar que la importancia del modelo Ising recae en que, a pesar de su fracaso unidimensional, fue de los primeros modelos capaces de usar un tratamiento mecanoestadístico de las transiciones de fase con éxito, de ahí su popularidad e importancia para la ciencia. [1].

Queremos estudiar el comportamiento de un sistema bidimensional de $N \times N$ spins sometidos a un campo magnético constante de magnitud B . Si consideramos que cada interacción spin-spin tiene el mismo valor, entonces el Hamiltoniano del sistema entonces está dado por [2]:

$$\mathcal{H} = -J \sum_{i,j} \sigma_i \sigma_j - B \sum_i \sigma_i \quad (1)$$

donde la parte de la izquierda da cuenta de la energía de interacción y la de la derecha es la asociada a la energía de la interacción spin-campo magnético. Pues, como sabemos [3],

$$U_{mag} = -\vec{\mu} \cdot \vec{B} \quad (2)$$

Por otro lado, cabe aclarar que la parte izquierda del Hamiltoniano del sistema es en realidad una simplificación

clásica para el acoplamiento entre dos spines dado por [4]:

$$\mathcal{H}_{spin} = -J \vec{S}_i \cdot \vec{S}_j \quad (3)$$

que desarrollado queda:

$$\mathcal{H}_{spin} = -J(S_i^x \cdot S_j^x + S_i^y \cdot S_j^y + S_i^z \cdot S_j^z) \quad (4)$$

Asumiendo una interacción débil en las componentes x e y llegamos al valor que aparece en el Hamiltoniano del Modelo Ising.

Notar que la resolución analítica de este sistema para cualquier valor realista en la cantidad de spines es irresoluble pues el espacio de Hilbert del sistema tiene $2^{N \times N}$ dimensiones. Lo que haremos entonces es plantear una solución que asuma solo una interacción local entre los spines y sus vecinos de modo que la interacción con el resto de la malla se tomará como despreciable. Se considerará como vecinos más cercanos a los spines que se encuentren justo arriba, abajo, a la izquierda y a la derecha del spin en cuestión.

Además, asumiremos que estamos trabajando una malla de spins infinita. De este modo evitaremos incluir los efectos de borde que van más allá del estudio del efecto fundamental de interés, las transiciones de fase magnéticas que se observan en materiales ferromagnéticos.

El Hamiltoniano que solo considera las interacciones locales, con el cual trabajaremos, tendrá entonces la siguiente forma:

$$\mathcal{H}_{loc,k} = -J \sigma_k \sum_{j \in \mathcal{D}} \sigma_j - B \sigma_k \quad (5)$$

para el k -ésimo spin de la red y considerando que $\mathcal{D}$ son los spines vecinos a este k -ésimo spin. Notar que en este caso se satisface que $\mathcal{H} = E$.

2. Metodología

Debido a la dificultad del problema original, hemos tomado ciertas aproximaciones que nos llevan a una expresión para la energía local de cada spin de la red.

A continuación lo que haremos será mostrar por qué esto nos permite modelar el sistema con métodos de MonteCarlo, ya sea Metropolis o Wolf.

Notar como, a partir de la ecuación del hamiltoniano local (5), podemos ver que el cambio de la energía de un spin cualquiera de la red está dado por:

$$\Delta E_k = 2\sigma_k \cdot \mathcal{H}_{loc,k} \quad (6)$$

pues, dada la aproximación de nula interacción de las componentes x e y del spin, σ_k solo posee dos posibles valores y estos son antisimétricos.

Luego, como nos interesa calcular propiedades macroscópicas a partir de las orientaciones de los spins, un tratamiento estadístico del problema es lo apropiado. El tema está en que resolver el problema de este modo nos requiere encontrar la función partición del sistema para encontrar las probabilidades de que el sistema se encuentre en un estado u otro, tal que la probabilidad de que el sistema esté en un estado ϵ_s está dada por [1]:

$$\mathcal{P}(\epsilon_s) = \frac{e^{-\mathcal{H}(\epsilon_s)/\tau}}{Z} \quad (7)$$

donde Z es la función partición del sistema, dada por:

$$Z = \sum_s e^{-\mathcal{H}(\epsilon_s)/\tau} \quad (8)$$

Evidentemente este calculo implica resolver el Hamiltoniano. Por ende, en vez de calcular explícitamente la probabilidad de que el sistema esté en un estado u otro, y desde ahí calcular propiedades macroscópicas como la magnetización, lo que haremos será calcular las probabilidades de transición de un estado a otro, dadas por:

$$\frac{\mathcal{P}(\nu)}{\mathcal{P}(\mu)} = \frac{e^{-\mathcal{H}(\nu)/\tau}}{e^{-\mathcal{H}(\mu)/\tau}} = e^{-\Delta\mathcal{H}/\tau} \quad (9)$$

Ahora bien, juntando la fórmula anterior con el cambio de energía que obtuvimos, bajo la aproximación local, para la energía de un spin, (6), tenemos entonces una forma de calcular la probabilidad de que un spin cambie su orientación en función de la orientación de sus vecinos más cercanos.

Notar que la energía total del sistema entonces solo depende del estado anterior de sistema, es decir, se trata de un sistema Markoviano. Es esto último lo que nos permite, entonces, modelar el sistema con algún método de MonteCarlo [5].

Primero modelaremos el sistema usando el algoritmo de Metrópolis. Para ello, seguiremos los siguientes pasos:

1. Primero generaremos una red de spins con orientaciones aleatorias, obtenidas a partir de la función `numpy.random.default_rng()` de Numpy.

2. Luego, iremos a una ubicación de la red al azar, calculamos la energía de ese spin y cambiaremos el spin de esa posición para ver como cambia la energía local ΔE_k .

3. Ahora aplicamos el criterio de selección:

- $\Delta E_k > 0$, entonces aceptamos el cambio si, y solo si, al generar un numero pseudoaleatorio p se cumple que $p < e^{-\Delta E_k/\tau}$. En caso contrario, devolvemos el spin a su posición original.
- $\Delta E_k \leq 0$, aceptamos el cambio pues la energía ha disminuido.

4. Elegimos una nueva posición al azar usando una función generadora pseudoaleatoria y repetimos 2 una cantidad N^2 veces, para serciorarnos de que, en promedio, toda la malla haya sido recorrida.

Una vez terminado el algoritmo decimos que se ha cumplido una iteración del MonteCarlo. Para obtener un resultado confiable el método se debe iterar la mayor cantidad de veces posibles pues cada barrido, que corren uno sobre la malla final del anterior, tiende a acercarse más a la distribución de probabilidades reales de los estados del sistema tal que, como sabemos, los procesos Markovianos tienden a satisfacer el principio de blance detallado [5]:

$$\mathcal{W}_{ij}\mathcal{P}_{eq}(S_i) = \mathcal{W}_{ji}\mathcal{P}_{eq}(S_j) \quad (10)$$

donde $\mathcal{W}_{rt}$ se define como la probabilidad de transición del estado r al estado t $\mathcal{W}_{rt} \equiv \mathcal{P}(S_r \rightarrow S_t)$

En particular, el proceso markoviano tarda en tender a un estado útil por ende antes de comenzar a ejecutar el analisis de los resultados de cada iteración de MonteCarlo se simula un porcentaje sin obtención de datos, para que el sistema llegue a un mínimo de cercanía a la distribución real. Estas corridas iniciales luego se descartan, en el código las llamaremos batches inactivas.

Debido a la gran cantidad de iteraciones necesarias para que el método converja es conveniente construir el código de manera vectorizada. Para esto se usará Numpy.

Una vez construído este código, se exploró e implementó el algoritmo de Wolf. La razón de cambio es que este algoritmo, a diferencia de Metropolis, trabaja con los dominios completos de spins en vez de ir chequeando uno por uno, por ende converge de mejor manera y, sobre todo, no e estanca en temperaturas cercanas a la temperatura crítica del sistema. La diferencia fundamental entre Metropolis y Wolf es que estos calculan las probabilidades de transición. Wolf no hace un análisis local de la energía. En cambio, el algoritmo de Wolf usa como probabilidad de transición la probabilidad de añadir o no un spin al cluster sobre el que actúa, para finalmente voltear el cluster completo. A continuación se entenderá en mayor detalle. Por ahora notar que esta probabilidad está dada por:

$$\mathcal{P}_{bond} = 1 - e^{-2J/\tau} \quad (11)$$

El algoritmo de Wof entonces está dado de la siguiente manera:

1. Primero generaremos una red de spins con orientaciones aleatorias, obtenidas a partir de la función `numpy.random.default_rng()` de Numpy.
2. Luego, iremos a una ubicación de la red al azar y nos moveremos a alguna casilla adyacente con el objetivo de formar un grupo de spins adyacentes, un cluster.
3. Si el vecino adyacente tiene una dirección contraria, no se añade el spin. En caso contrario, se genera un número pseudoaleatorio q y se evalúa si $q < 1 - e^{-2J/\tau}$. Si el numero generado es menor, entonces se une el spin al cluster.
4. Nos movemos a otra casilla adyacente y repetimos el procedimiento.
5. Una vez ya no queden vecinos por visitar, se cambian de dirección todos los spines del cluster
6. Nos movemos a una nueva posición aleatoria y repetimos desde el punto 2.

Ya que este algoritmo cambia la dirección de grupos de spins a la vez, evita quedarse estancado en temperaturas cercanas a la temperatura de Curie. Notar entonces como la diferencia principal está en que Metropolis contiene la probabilidad de funcionamiento del método de MonteCarlo en un análisis local de cambios de la energía al cambiar los spines uno a uno. Wolf, en cambio, contiene la probabilidad del MonteCarlo en la creación de cluster, los que si o si, una vez finalizado el analisis, se dará vuelta.

2.1. Calculo de cantidades Macro

Una vez completada la construcción de los algoritmos de Metropolis vectorizado y Wolf, para asegurar la correcta termalización para cada valores de entrada (Magnitud del Campo Magnético B , Valor de la Constante de acoplamiento J y el número de partículas $N \times N$) se implementó un sistema de termalización por autocorrelación. Este está basado en el coeficiente de correlación de Pearson que, como sabemos, mide la correlación entre dos canidades, en este caso una misma cantidad pero medida en diferentes tiempos, aproach clásico usado para series de este estilo (series estacionarias).

$$C(t) = \frac{Cov(M(t_0), M(t_0 + t))}{\sqrt{Var(M(t_0))}\sqrt{Var(M(t_0 + t))}} \quad (12)$$

Con

$$Cov(X, Y) = (X - \langle X \rangle)(Y - \langle Y \rangle) \quad (13)$$

Una vez lista la autocorrelación simplemente se definió un criterio, basado en la estabilidad de la autocorrelación al aumentar el número de corridas, para encontrar el punto desde el cuál se comenzarían a considerar las muestras resultantes del Montecarlo, Metropolis o Wolf, es decir, nuestros llamados "bins activos".

Con el paso anterior ya cerramos toda la maquinaria necesaria mínima para empezar a calcular cantidades macroscópicas de manera fiable a partir de nuestro código

basado en MonteCarlo. Justamente eso fue lo que se hizo a continuación. Primero que nada se calculo la magnetización y energía promedio usando:

$$\langle M \rangle = \frac{\sum_i \sigma_i}{N^2} \quad (14)$$

$$\langle E \rangle = -\frac{J}{2N^2} \sum_{i,j} \sigma_{i,j} \cdot (\sigma_{i+1,j} + \sigma_{i-1,j} + \sigma_{i,j+1} + \sigma_{i,j-1}) \quad (15)$$

donde σ_i representa el valor, ± 1 , del i -ésimo spin de la malla, con i recorriendo desde 1 hasta N^2 y N^2 el número total de spines de la red.

Una vez calculada la magnetización, se pueden calcular ciertas cantidades macroscópicas derivadas de lo anterior. Las que nos interesan en este caso son la suceptibilidad χ y el calor específico del sistema C , cada uno con su repectiva formula:

$$\langle \chi \rangle = \frac{N^2}{T} (\langle M^2 \rangle - \langle M \rangle^2) \quad (16)$$

$$\langle C \rangle = \frac{N^2}{T^2} (\langle E^2 \rangle - \langle E \rangle^2) \quad (17)$$

Notar, en particular, que el código se diseñó de tal forma que estas cantidades macroscópicas se obtienen para multiples T , así, permitiendonos estudiar la evolución de estas cantidades al cambiar la temperatura, específicamente de interés en temperaturas cercanas a la temperatura de curie T_c . De esta forma, lo que se obtuvo fueron las listas $\langle M \rangle (T)$, $\langle \chi \rangle (T)$ y $\langle C \rangle (T)$.

Brevemente, el código diseñado usa la formula de correlación ya mencionada para luego a partir de ella encontrar el punto desde donde comenzar a medir y, desde ahí, usa el algoritmo correspondiente para resolver la estadística de la malla, ya sea Metropolis vectorizado o Wolf, para finalmente guardar el calculo obtenido para M y E en una lista que posteriormente se usa para calcular los valores promedios para una temperatura en particular. Por último, repite lo anterior para diversas temperaturas y los resultados los va guardando en una lista, desde donde gráfica los resultados.

Notar que, evidentemente, al ser MonteCarlo un método probabilístico, este tiene intrínsecamente un error asociado a los datos obtenidos a través del método. Para cuantificar ese error correremos un remuestreo con bootstrapping sobre la data ya simulada con MonteCarlo, ya sea Metropolis o Wolf, para así establecer intervalos de confianza para las mediciones macroscópicas ya mencionadas.

Lo anterior implica, por supuesto, añadir al código alguna función que de cuenta del remuestreo. En el código es *bootstrap error()*

3. Resultados Preliminares

Al correr el código con termalización basada en la autocorrelación del muestreo se obtuvo una convergencia en

la estabilidad de la autocorrelación para toda temperatura bajola temperatura de curie teórica, obtenida desde el análisis de Onsager y llamada como T_c en el código. Esto ocurre justamente por el hecho de que cerca de esa temperatura los spines oscilan violentamente, pues se encuentran en la fase desordenada, lo que no permite que el promedio macroscópico converja a algún valor. Lo anterior es justamente lo encontrado analíticamente por Onsager.

Cabe recalcar que se encontró una mejor estabilidad para el algoritmo de Wolf versus el de Metrópolis cuando aumenta el número de spines relativo al número de corridas del MonteCarlo. En el código actual se trabajó principalmente con $N = 20$ y 4000 corridas, ya sean Metropolis o de Wolf. Para N más grandes se encontraron resultados que no describen lo que uno ve experimentalmente, con barras de error muy grandes, y ello es, principalmente, debido a que no se alcanza el tiempo de termalización del MonteCarlo dentro de las 4000 corridas. Además, se encontró que para números muy bajos de N y altas corridas relativas a ella, Metropolis describe el fenómeno de manera más *suave*. Esto, principalmente debido a la naturaleza de *cluster* del algoritmo de Wolf que rápidamente se alcanza en esos casos, generando así una oscilción, que si bien se promedia correctamente en las cantidades macro, que muestra gráficas no tan suaves para la magnetización, esta oscila de manera violenta pero converge de todas maneras.

4. Estado del Proyecto

Tal como se mencionó en el último avance, ya se arregló la implementación del algoritmo de Wolf, se automatizó la asignación de batches activos e inactivos basado en la autocorrelación de los datos obtenidos y se implementó todo lo necesario para calcular cualquier cantidad macroscópica de interés, pues se implementó una metodología apropiada para la cuantificación del error de cada medición macroscópica obtenida, a través de un remuestreo por booststrap. En particular, se calculó la evolución de la magnetización, la capacidad calórica y la susceptibilidad magnética del sistema para diversas temperaturas.

El siguiente paso es ser capaces de calcular cantidades un siguiente nivel arriba en la jerarquía de caracterización del material. Pues primero estudiamos y simulamos la interacción entre los elementos fundamentales del modelo, spins. Con ello, y la teoría/modelación adecuada, fuimos capaces de calcular cantidades macroscópicas del sistema. Ahora queremos usar los cálculos de estas diversas cantidades macroscópicas para obtener la propiedad fundamental que caracteriza el fenómeno, el exponente crítico ν . Este determina si existe o no transición de fase y donde ocurrirá, manifestando su efecto medible en la temperatura de Curie del sistema. Para obtenerla necesitaremos los resultados de las cantidades macroscópicas para diversas temperaturas y luego, a partir de ajuste, extraer el exponente crítico. El detalle de esto es que los modelos que hasta ahora hemos construido son todos finitos y el exponente crítico es una cantidad calculada teóricamente para arreglos infinitos. Es por ello que será necesario hacer el colapso adecuado de los datos usando *Finite-Size Scaling*.

Esto y construir una animación que muestre los resultados será la siguiente y última fase del proyecto.

Referencias

- [1] C. Kittel y H. Kroemer, *Thermal Physics*, 2da ed., San Francisco: W. H. Freeman, 1980.
- [2] S. K. Ma, *Statistical Mechanics*, Singapur: World Scientific, 1985.
- [3] D. J. Griffiths, *Introduction to Electrodynamics*, 4ta ed., Pearson, 2013.
- [4] D. J. Griffiths y D. F. Schroeter, *Introduction to Quantum Mechanics*, 3ra ed., Cambridge University Press, 2018.
- [5] D. P. Landau y K. Binder, *A Guide to Monte Carlo Simulations in Statistical Physics*, Cambridge: Cambridge University Press, 2014.